SGSzero: Training a Compact Sanguosha Agent via Privileged Rule-Based Supervision

Evan Wen
Independent Researcher

Abstract

An agent that runs on a player's own machine has to decide from the information that player can see, and it has to decide quickly enough to keep up with the game. These two requirements are easy to state and awkward to satisfy at the same time, because the information that makes training easy is normally the information that deployment removes.

We ran into this problem while training a Sanguosha agent. Our handcrafted rule engine scores every card in hand by its utility. Supplying those scores to a small Transformer as input features produced a model that reached 83.9% accuracy on held-out data and 35.3% when we replayed the same checkpoint with the deployment tokenizer, which never emits the scores; it had learned to read the score rather than the position. Distilling the scores into an auxiliary head, trained alongside the policy and removed before deployment, recovers most of the loss: 80.9% on the same sub-task with 1.76M parameters and CPU inference.

What we did not expect was where the gains came from. Hyper-parameter search was unproductive: seven configurations all landed between 72% and 73%, whereas a 7.6% increase in the training set moved accuracy by 5.3 points. Most remaining errors (89%) fall below 0.9 confidence, boundary calls rather than systematic misunderstandings of the game. And the aggregate metric concealed a weak subgroup, hands without an attack card, where the model sits 17.2 points above baseline once valuation supervision is present and the auxiliary head adds a further 3.6.

1 Introduction

Card game AI has produced a series of landmark results: AlphaGo and AlphaZero for perfect-information board games [1, 2], Libratus and Pluribus for poker [3, 4], Suphx for mahjong [5], and DouZero for DouDizhu [6]. Nearly all of this work was developed under two assumptions that a practical assistant cannot make.

The first is that the agent may observe the full game state internally, regardless of what a human player could see. The second is that compute at decision time is abundant. Deployed assistants and training tools for human players satisfy neither condition: they must decide from exactly the information a human holds, and if they are to run alongside a live game client, or on a phone, inference has to complete in milliseconds on a CPU.

These constraints interact, and the interaction creates a genuine learning problem. A rule-based engine written by a domain expert does have access to internal valuations: it scores each card in hand by how useful that card would be. Those valuations are highly informative for the decision and unavailable at deployment. The obvious approach, feeding them to the model as an input feature, produces a model that learns to read the number instead of learning the game. It performs well in-distribution and fails once the feature is absent.

We encountered this failure in its sharpest form. A model trained with valuations as input scored 81.4% and 83.9% on two seeds over a held-out split, and the better of the two checkpoints scored 35.3% when we

re-ran it on the same split with the valuation tokens stripped from the input, which is the only information a deployed copy would have.

Our contribution is a practical recipe that resolves this, together with an empirical study of what actually drives performance in this regime. We formulate the Sanguosha decision problem under a human-observable information constraint and release a state encoding that matches what a player actually sees (Section 3). We then separate two kinds of privileged signal that are often conflated: closing an *observability* gap, as perfect-information distillation does [7], and closing a *knowledge* gap, as here. The distinction matters in practice, not only in terminology, because revealing hidden state cannot close a knowledge gap; a network that can see every card still has to learn what a card is worth. Our method applies LUPI [10, 11] as an auxiliary card-value head trained on the engine's valuations and discarded before deployment, which recovers most of the gap (78.0–78.4% to 80.8–81.8%) at no inference cost (Sections 4–5). We also report our negative and diagnostic results in some detail, because in this setting they proved more informative than the headline number (Section 6).

2 Related Work

Card and board game AI. AlphaZero [2] combines Monte Carlo tree search with a policy-value network trained purely by self-play. DouZero [6] trains a Deep Monte-Carlo value network for DouDizhu with a compact representation and no human data. Suphx [5] reaches superhuman mahjong through self-play with global reward prediction. Our setting differs in two respects: the game involves chained skill effects rather than a fixed action vocabulary, and we explicitly forbid information that is unavailable to a human player.

Training with information unavailable at deployment. The closest line of work to ours is PerfectDou [7], which introduces *perfect information distillation*: agents are trained as if the game were fully observable, with the network seeing every player's hand, and the resulting policy is then executed under partial observability. This is a *perfect-training, imperfect-execution* framework, and it shares our high-level philosophy that privileged signals available only during training can substantially improve a deployable policy.

The two settings differ in what the privileged signal is. In PerfectDou it is a part of the game state that objectively exists but is hidden from the player, namely the opponent's cards, so distillation removes an *observability* gap. In our setting it is the rule engine's scalar valuation of each card, a quantity that exists nowhere in the game state and is produced by an external symbolic program. What we remove is a *knowledge* gap. A perfect-information formulation cannot express it, since even with all cards revealed a network still has to learn what a card is worth.

The two approaches are therefore complementary rather than competing. PerfectDou could in principle be layered on top of our method, so that the agent both observes everything and receives valuations, and our contribution is orthogonal to it: how to inject a symbolic expert's internal valuation into a compact neural policy without letting that valuation dominate the learned representation.

Privileged information. Learning Using Privileged Information (LUPI) was introduced by Vapnik and Vashist [10] and extended in [11]: a teacher has access to additional information x^* at training time that is unavailable at test time. Generalized distillation [8] places this and Hinton-style distillation in one framework, with LUPI as the case where the extra channel is a representation rather than a softened label, and industrial recommenders train a privileged teacher and ship only the student [9]. The failure mode we hit, where the shared representation comes to depend on the privileged channel and the student loses the primary features, follows from that dependence rather than from any particular implementation: if the privileged signal is cheap to read, gradient descent will read it. Our results reproduce it, and a low-weight auxiliary head avoids it because the head supervises a separate output instead of feeding the input.

Included (human-observable)	Excluded (engine-internal)
Own hand: card name, rank, suit	Card utility score from the rule engine
All heroes' skills (public)	Internal event stream
All players' HP and max HP	Turn index, seat index
All identities (public in this mode)	Cards already played this round
Opponents: hero, HP, hand *count*, identity, alive flag	Opponent hand *contents*
Deck remaining *count*	Top card of deck
Equipments, judgement area, chained/flipped status, marks	
Current phase	

Table 1: State features. The left column is what a human player can observe; the right column is available to the engine but withheld from the model, because a human cannot see it.

Compact models for structured decisions. Transformers [12] over tokenized state sequences have become a standard way to encode heterogeneous game state. Our encoder is deliberately small (1.76M parameters) to satisfy the deployment constraint, following the broader argument that domain-specialized compact models can outperform far larger general models on narrow tasks [13].

3 Problem Setup

3.1 The Game

Sanguosha (Legends of the Three Kingdoms) is a multi-player Chinese card game. Each player controls a hero with a unique skill, holds a hand of cards, and acts in turn. Effects chain: playing a card may trigger skills, which may trigger further skills, and resolution order matters. We study the two-player variant implemented by the open-source client we use as an environment, in which identities are public and only the top of the deck is hidden.

We focus on **Task A**: at the play phase, decide whether to play a card or pass. This is the highest-frequency decision in the game and a prerequisite for the harder *which card* task.

3.2 Human-Observable State

The input is a token sequence. Table 1 lists what is included and, more importantly, what is excluded.

The resulting sequence has a vocabulary of 561 tokens and a 95th-percentile length of 57 tokens over the collected records (54 on the validation split). Structural tokens `[BOS]`, `[SELF]`, `[HAND]`, `[OTHER]` and `[EOS]` delimit segments.

3.3 Privileged Signal

The rule engine we use as a teacher is a 10,966-line handcrafted policy. For each card in hand it exposes a scalar utility $v \in \{0, \ldots, 12\}$. This is the kind of information a strong human player forms implicitly and a rule engine computes explicitly, and it is precisely what is unavailable at deployment.

4 Method

4.1 Encoder

A token embedding of dimension $d = 256$ feeds three Transformer encoder layers (4 heads, feed-forward 512, pre-norm, dropout 0.1). The decision representation is taken from the `[SELF]` position:

$$h = \mathrm{LayerNorm}\big(\mathrm{Encoder}(x)_{[\mathtt{SELF}]}\big), \qquad \hat{y} = \mathrm{softmax}(Wh + b). \tag{1}$$

We use the `[SELF]` position rather than mean pooling deliberately. Mean pooling over a variable-length hand produces representations whose scale depends on hand size, conflating how many cards are held with which cards they are. The `[SELF]` token attends over the whole sequence and therefore carries an aggregate summary whose scale is independent of hand size. Total parameters: 1,760,527.

4.2 Privileged Value Head

During training only, we attach an auxiliary head that predicts the engine's utility bucket for each card in hand:

$$\mathcal{L} = \mathcal{L}_{\mathrm{dec}} + \lambda\, \mathcal{L}_{\mathrm{val}}, \qquad \mathcal{L}_{\mathrm{val}} = \frac{1}{|H|} \sum_{i \in H} \mathrm{CE}\big(\mathrm{MLP}(x_i),\ \mathrm{bin}(v_i)\big), \tag{2}$$

where H is the set of card positions in the hand and v_i is the teacher's utility for card i, discretized into 13 bins. We use $\lambda = 0.5$.

The head is discarded at deployment. Its purpose is representational: by requiring the encoder to predict card utility from card identity, rank and suit alone, it forces the encoder to internalize a utility function rather than read one from an input feature.

This is the LUPI recipe applied to a rule engine rather than to a human expert. The teacher is symbolic and cheap, the student neural and fast.

5 Experiments

5.1 Data

Training data is generated by letting the rule engine play against itself in the live game client, logging (state, action) pairs. The latest dataset holds 13,536 records with the full observable feature set; restricting it to play-phase decisions, the sub-task evaluated here, leaves 9,810 samples, split by game ID into 9,470 training and 340 validation records with no game appearing in both splits. An earlier corpus of 78,536 records, collected before the observable-only encoder existed, supplies the two data-scaling comparisons in Section 6; where a number comes from that corpus we say so. That corpus is still growing, so the figure is a snapshot taken on 30 October 2026.

5.2 Baselines

We report the majority-class baseline on each validation split, that is, the accuracy of always predicting the more frequent label. A model that learns only the label prior scores exactly this, so it is the floor against which the other rows should be read. The baseline is 68.0% on the 819-record split and 74.4% on the 340-record split; the two differ in class balance enough that accuracies across them are not directly comparable.

Configuration	Input	Val size	Accuracy	macro-F1
V5 (privileged input, 2 seeds)	full state + v, u	819	81.4–83.9%	0.799–0.825
same, deployment tokenizer	observable only	819	**35.3%**	—
V6 (observable only, 2 seeds)	observable	819	78.0–78.4%	0.760
V7 (+ privileged head, 2 seeds)	observable	819	80.8–81.8%	0.786–0.796
V8 (+ full fields, + head, 2 seeds)	observable	340	**80.6–80.9%**	0.756–0.762
Majority baseline	—	340	74.4%	—
majority baseline, 819 split	—	819	68.0%	—

Table 2: Accuracy across configurations. V5 shows the failure mode: high in-distribution accuracy collapses when the privileged tokens are removed, measured by re-running the V5 checkpoint on the same validation split with the deployment tokenizer. V7 and V8 recover most of the gap while remaining deployable.

5.3 Main Results

Table 2 summarizes the progression. The informative comparison is between V5 and V7. V5 is more accurate in-distribution but unusable, since it depends on tokens that do not exist at deployment; replaying the checkpoint with the deployment tokenizer costs 48.6 points, from 83.9% to 35.3%. Removing only the valuation tokens while leaving the other engine-internal tokens in place gives 38.5%, so most of the collapse is attributable to the valuation channel rather than to the remaining internal fields. V7 gives up 2.1 points of in-distribution accuracy and becomes deployable.

V8 adds observable fields that had been missing (chained/flipped status, marks) and reaches 80.9%, statistically indistinguishable from V7 despite a smaller validation set and a different class balance. We read this as evidence that the new fields entered the representation without displacing anything, though the two splits are too different and too small to support a sharper claim.

Two of these comparisons carry enough data to test, and they behave differently. The auxiliary head gains +3.4 points over V6 on the same 819 validation games. It flips 52 predictions from wrong to right and 80 from right to wrong, which McNemar's test puts at $p = 0.018$. The unpaired 95% Wilson intervals, however, overlap ([75.4, 81.1] against [79.0, 84.3]); a reader comparing intervals alone would reach the opposite conclusion. The pairing is what makes the difference visible. The V8 comparison is weaker. Its 340 records give a 95% interval of [76.4, 84.7], wide enough to contain the point estimates of both V6 and V7, so the claim that the new fields cost nothing is a null result rather than a demonstration of equivalence, and we do not lean on it.

5.4 Deployment Cost

The model has 1,760,527 parameters (6.72 MB in fp32). Training for 40 epochs takes 70 seconds on a single RTX 4060 Laptop GPU. Inference is one forward pass over a sequence of at most about 100 tokens and runs on commodity CPUs in milliseconds, well inside a human player's decision window.

6 Analysis

Sections 6.1 and 6.2 draw on the earlier 78,536-record corpus, and we flag the confound that matters in each. Sections 6.3–6.5 use the shipped checkpoints and their own validation splits.

Configuration	Accuracy	macro-F1
d=128, L=2, lr 3e-4	72.3%	0.702
d=128, L=2, lr 1e-3	72.0%	0.692
d=256, L=3, lr 3e-4	73.0%	0.692
d=128, L=4, lr 3e-4	72.0%	0.695
d=192, L=3, lr 2e-4	72.8%	0.697
d=256, L=3, *more data*	**78.3%**	0.756

Table 3: Hyper-parameter sweep versus data scaling. Model size, depth and learning rate make little difference in this regime; the data change does, though the validation split moved with it.

6.1 Capacity Is Not the Bottleneck; Data Is

We swept seven hyper-parameter configurations (width $\in \{128, 192, 256\}$, depth $\in \{2, 3, 4\}$, learning rate $\in \{2\text{e-}4, 3\text{e-}4, 1\text{e-}3\}$); Table 3 shows five representative runs. Every configuration landed between 72.0% and 73.0%. Increasing the training set from 17,830 to 19,188 records (+7.6%) under a *fixed* configuration moved accuracy from 73.0% to 78.3% (+5.3 points). That comparison is not clean: the two runs also used different validation splits, so part of the 5.3 points is not attributable to data alone, and repeated seeds on the larger corpus spread over 75.6–78.3%.

For a project of this size the practical implication is unwelcome. We tuned seven configurations over several days and gained half a point; a few hours of data collection gained five. The binding constraint was the quantity and quality of logged decisions, not the architecture search.

6.2 Data Quality Dominates Quantity

On the same early corpus, 11,084 records carrying the complete feature set matched 29,834 mixed records, at 78.4% and 77.4% respectively. The two runs used different validation splits (453 and 1,436 records) and the gap is inside the noise either way, so the defensible reading is that a third of the data matched, not that it beat. On that reading one clean record carries about as much information as 2.7 mixed ones. The reason is distributional: 64% of the mixed corpus lacked opponent-action features, so the model was trained on a distribution in which the same input sometimes carried information and sometimes did not.

6.3 Errors Are Boundary Cases, Not Misconceptions

For the shipped V8 model on its 340-record split, 65 predictions are wrong: 34 with $p < 0.7$, 24 with $0.7 \leq p < 0.9$, and 7 with $p \geq 0.9$. Calibration is close to diagonal; predictions made at $p \geq 0.95$ (103 of 340) are correct 96.1% of the time. The model is therefore not confused about the game at large but uncertain at close decisions, and the seven confident errors are the ones worth reading by hand.

6.4 A Weak Subgroup Hidden by the Aggregate

Splitting the 819-record validation set by whether the hand contains an attack card (a *sha*) isolates the decisions where card valuation is the whole question. Table 4 compares V6 and V7 on that split, so the only thing that changes between the last two columns is the auxiliary head.

When no attack card is held, the decision reduces to whether to spend a trick card, an equipment or a peach, which is a question of card value alone, and the two subgroups sit 21.8 points apart in baseline. The head's contribution is modest and consistent, +3.3 and +3.6 points. The larger effect on this subgroup comes from training on records where valuation supervision exists at all: on an earlier corpus collected before any

Subset	n	Baseline	V6 (no head)	V7 (with head)
Holds attack card	424	78.5%	82.6% (+4.0)	85.9% (+7.3)
No attack card	395	56.7%	73.9% (+17.2)	77.5% (+20.8)

Table 4: Subgroup accuracy on the 819-record validation split, baseline-relative. V6 and V7 differ only in the auxiliary head, so the difference between the last two columns is the head's own contribution: 3.3 points with an attack card and 3.6 without.

valuation signal was available, this subgroup sat at 63.1% against a 60.2% baseline, a 2.9-point margin, and reaches 77.5% here. Taken that way, most of the improvement is a data effect and the head supplies the remainder.

An oversampling control on the earlier corpus points the same direction: tripling the weight of the no-attack subgroup *lowers* accuracy (78.4% $\rightarrow$ 77.9%), so the deficiency is informational rather than statistical.

6.5 Headroom

On the 9,810 play-phase samples of the current dataset there are 9,794 distinct token sequences, and 22 samples (0.22%) fall into groups where one sequence appears with both labels, implying a Bayes-optimal ceiling near 99.8%. Almost every position is unique, so the model must generalize rather than memorize. The gap between 81% and 99.8% is headroom, not label noise.

6.6 Validation Protocol

We ran 13 checks before trusting any number, among them single-batch overfitting (which reaches 100%), a shuffled-label control (67.7% against a 68.4% baseline, so no leakage), padding-mask consistency (6×10^{-8} difference between single and batched inference), UNK rate (0.0066% on the current corpus) and game-level train/validation disjointness (zero overlap). The shuffled-label and mask-consistency checks are worth treating as mandatory in a pipeline of this kind, since both are easy to skip and both catch silent bugs.

7 Limitations

Our evaluation is restricted to Task A (play or pass) in the two-player mode. The *which card* task, response decisions and the multi-player identity mode are not yet evaluated. Validation sets are small (340–819 records); the no-attack subgroup now rests on 395 records, enough to see a direction but not to put a tight interval on the head's contribution, and the 340-record split supports only coarse comparisons. Two analyses (Sections 6.1, 6.2) and the oversampling control come from an earlier corpus and are confounded with the validation-split changes we disclose there.

The most important limitation is what we measure. Every accuracy in this paper is agreement with the teacher's action on a held-out game, not playing strength. An imitator can agree with its teacher frequently and still be worse at winning games, because agreement counts all decisions equally while games are decided at a few of them. We have not run the agent against the engine, so we cannot state a win rate, and the 80.9% headline should be read as an imitation score rather than as evidence that the agent plays well. Doing this properly requires a match protocol, a seat-balance correction and enough games to put an interval on the win rate; it is the next experiment we intend to run.

The teacher is a handcrafted engine, so the student inherits its strategic ceiling until self-play improvement is added. We regard self-play fine-tuning as the natural next step and as the point at which the "zero

human data" claim becomes fully accurate. Finally, the game client we instrument is a third-party open-source implementation, and results may not transfer verbatim to the commercial client.

8 Conclusion

The hard part of this problem turned out to be the gap between what a rule engine knows and what a player can see, rather than the game itself. A rule engine's internal valuations cannot be used at deployment, but they can be used at training time as privileged information, provided they supervise an auxiliary head instead of entering as features. Feeding them in directly produces a model that reads numbers instead of playing the game and loses 48.6 points when the tokens are withdrawn.

Model capacity proved irrelevant in this regime, data cleanliness mattered more than volume, and aggregate accuracy concealed a subgroup whose margin over baseline depends on valuation supervision, to which the head adds a further 3.6 points. The resulting model has 1.76M parameters, trains in 70 seconds on a consumer GPU and runs inference on a CPU.

We will release the state encoding, training pipeline and validation protocol with the next revision of this preprint, and we intend to release a self-play improved agent together with a benchmark for Sanguosha decision-making. Until then, the validation protocol in Section 6 is specified precisely enough to be reproduced from this text alone, and we will share checkpoints and logs on request.

Data and Code Availability

The game client used as an environment is a third-party open-source implementation and is not redistributed here. The logged (state, action) pairs derive from it and are subject to its licence. The tokenizer, the training script and the thirteen validation checks are ours and will be published as a single repository; no part of the reported numbers depends on undocumented private tooling beyond the client itself.

Acknowledgment

We thank the maintainers of the open-source Sanguosha client used as our environment, and the authors of the handcrafted rule engine that served as our privileged teacher.

References

[1] D. Silver, A. Huang, C. J. Maddison, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

[2] D. Silver, T. Hubert, J. Schrittwieser, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.

[3] N. Brown and T. Sandholm. Superhuman AI for heads-up no-limit poker: Libratus beats top professionals. *Science*, 359(6374):418–424, 2018.

[4] N. Brown and T. Sandholm. Superhuman AI for multiplayer poker. *Science*, 365(6456):885–890, 2019.

[5] J. Li, S. Koyamada, Q. Ye, et al. Suphx: Mastering Mahjong with deep reinforcement learning. *arXiv:2003.13590*, 2020.

[6] D. Zha, J. Xie, W. Ma, et al. DouZero: Mastering DouDizhu with self-play deep reinforcement learning. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.

[7] G. Yang, M. Liu, W. Hong, W. Zhang, F. Fang, G. Zeng, and Y. Lin. PerfectDou: Dominating DouDizhu with perfect information distillation. *arXiv:2203.16406*, 2022.

[8] D. Lopez-Paz, L. Bottou, B. Schölkopf, and V. Vapnik. Unifying distillation and privileged information. In *International Conference on Learning Representations (ICLR)*, 2016. *arXiv:1511.03643*.

[9] C. Xu, Q. Li, J. Ge, J. Gao, X. Yang, C. Pei, F. Sun, J. Wu, H. Sun, and W. Ou. Privileged features distillation at Taobao recommendations. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2020. *arXiv:1907.05171*.

[10] V. Vapnik and A. Vashist. A new learning paradigm: Learning using privileged information. *Neural Networks*, 22(5–6):544–557, 2009.

[11] V. Vapnik and R. Izmailov. Learning using privileged information: Similarity control and knowledge transfer. *Journal of Machine Learning Research*, 16:2023–2049, 2015.

[12] A. Vaswani, N. Shazeer, N. Parmar, et al. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[13] S. Gunasekar, Y. Zhang, J. Aneja, et al. Textbooks are all you need. *arXiv:2306.11644*, 2023.